E-COMMERCE LAB

Vanilla Pathway

HTML · CSS · JavaScript · PHP · MySQL

Technology Stack

View (HTML + CSS + JS): Browser-rendered pages, styling, and client-side interactivity

Controller (PHP): Handles HTTP requests, validates input, calls Model classes, outputs response

Model (PHP Classes): All database logic — CustomerClass, ProductClass, CartClass

Database (MySQL): shoppn schema — customers, products, brands, categories, cart, orders

---

> ⚠ **IMPORTANT:** *Labs build upon each other — this is a continuous flow, not independent exercises. Code plagiarism is checked automatically. Work independently.*

PROJECT OVERVIEW

This lab series guides you step by step through building a fully functional e-commerce web application. By the final task you will have a working store with customer registration, login, product management, a shopping cart, and payment — using the classic server-rendered stack of HTML, CSS, JavaScript, PHP, and MySQL.

Every PHP file has one clear responsibility. You will never mix database queries with HTML output in the same file by the time you are done.

The MVC Pattern — What It Means in This Project

Layer	Files	Responsibility
Model	`classes/*.php`	PHP classes that extend db_class. All SQL lives here. No HTML, no $_POST, no echo.
View	`views/*.php`	HTML templates with minimal PHP — only loops and conditional display. No database calls.
Controller	`controllers/*.php`	Receives the request, validates input, calls the Model, decides which View to load. The traffic director.
Core	`core/*.php`	Database connection, session helpers, routing utilities, shared config. Included once.
Actions	`actions/*.php`	Thin files that handle a single form submit (register, login, add to cart). Call a Controller method and redirect.

FOLDER STRUCTURE

Stick to this structure from Task 2 onwards. Every new file you create should have a clear home.

```
shoppn/
  core/
    db_class.php          ← PDO/MySQLi base class — connection only
    db_cred.php           ← DB credentials (host, user, pass, dbname)
    core.php              ← session_start(), CORS, shared helpers, error logger
  classes/
    CustomerClass.php     ← extends db_class; register, login, edit, delete
    ProductClass.php      ← extends db_class; CRUD for products, brands, categories
    CartClass.php         ← extends db_class; add, remove, update qty, totals
  controllers/
    CustomerController.php  ← instantiates CustomerClass, exposes methods to actions
    ProductController.php   ← instantiates ProductClass
    CartController.php      ← instantiates CartClass
  actions/
    register_action.php     ← processes registration form POST → controller → redirect
    login_action.php        ← processes login form POST → controller → redirect
```

```
    add_brand_action.php
    update_brand_action.php
    add_category_action.php
    update_category_action.php
    add_product_action.php
    update_product_action.php
    add_to_cart_action.php
    remove_from_cart_action.php
    update_qty_action.php
    process_payment_action.php
  views/
    layout/
      header.php          ← <head>, nav, search bar — included in every view
      footer.php          ← closing tags, copyright
      sidebar.php         ← categories & brands sidebar
    home.php
    register.php
    login.php
    all_products.php
    single_product.php
    search_results.php
    cart.php
    checkout.php
    payment.php
    payment_success.php
    payment_failed.php    ← optional
    account/
      my_account.php
      edit_account.php
      change_pass.php
      delete_account.php
    admin/
      brand.php           ← add/edit brands
      category.php        ← add/edit categories
      product.php         ← add/edit products
  css/
    style.css             ← your own stylesheet (or import a CSS framework)
  js/
    validate.js           ← client-side form validation with regex
    cart.js               ← optional: async cart update without full reload
  images/
    logo.gif
    ad_banner.gif
    products/             ← uploaded product images
    customers/            ← uploaded customer profile images
  error/
    error.log             ← your own error logger
  database/
    shoppn.sql
  index.php               ← entry point — includes core, loads home view
  logout.php
```

MVC: Notice the strict separation: classes/ never echo HTML. views/ never run SQL. actions/ never echo HTML — they only redirect. This is MVC.

WEEKLY TASKS

TASK 1: DATABASE SETUP | Due: Week 1

Objective

Import and verify the shoppn MySQL database that the whole project depends on.

Steps

- Import the provided SQL file using phpMyAdmin or the command line:

```
mysql -u root -p < database/shoppn.sql
```

- Confirm the following tables exist: brands, categories, customer, products, cart.
- Note the customers schema: customer_id (PK, auto), customer_name, customer_email (UNIQUE, max 50 chars), customer_pass, customer_country, customer_city, customer_contact, customer_image, user_role.
- Note the products schema: product_id (PK, auto), product_cat (FK), product_brand (FK), product_title, product_price, product_desc, product_image, product_keywords.
- Note the user_role when registering a new user — this controls admin vs customer access:

```
-- user_role is already included in shoppn.sql (DEFAULT 2 = customer, 1 = admin) -- No
ALTER TABLE needed — it is created with the import.
```

- Create core/db_cred.php with your local credentials:

```
<?php
  define('DB_HOST', 'localhost');
  define('DB_USER', 'root');
  define('DB_PASS', '');
  define('DB_NAME', 'shoppn');
?>
```

Tip: Never commit db_cred.php to a public repository. Add it to .gitignore if you are using Git.

TASK 2: FOLDER STRUCTURE & CORE FILES | Due: Week 1

Objective

Scaffold the project skeleton so every future file has a clear, correct home before you write any feature code.

Core files to create

- core/db_class.php — the database connection base class. All Model classes will extend this:

```
<?php
class Database {
  protected $conn;
  public function __construct() {
    require_once 'db_cred.php';
    $this->conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($this->conn->connect_error) {
      error_log($this->conn->connect_error);
      die('Connection failed.');
    }
  }
}
```

- core/core.php — included on every page. Starts the session, sets the timezone, requires db_class, defines shared helper functions like get_ip(), redirect($url), is_logged_in(), is_admin().
- index.php — the entry point. Requires core/core.php and loads views/home.php.
- views/layout/header.php — the shared page header with <head>, navigation bar, and search box. Included at the top of every view.
- views/layout/footer.php — closing HTML tags and copyright line.
- views/layout/sidebar.php — categories and brands sidebar. Fetches from DB via ProductController.
- logout.php — destroys the session and redirects to index.php.

Verify it works

- Browse to http://localhost/shoppn/ — the home page should load without PHP errors.
- Check the nav, header, and footer render correctly from the shared layout files.

MVC: header.php is the View's layout layer. It contains only HTML and minimal PHP for session-based nav items (e.g. show Logout if logged in). No SQL here.

TASK 3: CUSTOMER REGISTRATION | Due: Week 2

Objective

Build the full registration flow: View form → JavaScript validation → Action file → Controller → Model → DB.

Model — classes/CustomerClass.php

- Create CustomerClass extending Database.
- Method emailExists($email): SELECT customer_email FROM customer WHERE customer_email = ? — use a prepared statement. Return true/false.
- Method addCustomer($name, $email, $pass, $country, $city, $contact): INSERT using a prepared statement. Hash the password first with password_hash($pass, PASSWORD_BCRYPT) before passing it in.
- Never build SQL strings with string concatenation — always use prepared statements.

> ⚠ **SECURITY:** Raw $variables inside SQL strings — this is a SQL injection vulnerability. Use prepared statements for every query that includes user input.

Controller — controllers/CustomerController.php

- Instantiate CustomerClass. Expose a register($data) method that calls emailExists() then addCustomer().
- Return a structured result array: ['success' => true] or ['success' => false, 'error' => 'Email already registered'].

Action — actions/register_action.php

- Only runs on POST. Require core/core.php and CustomerController.php.
- Sanitise all inputs server-side: trim(), strip_tags(), validate email with filter_var($email, FILTER_VALIDATE_EMAIL).
- Validate field lengths against DB schema (e.g. customer_email is VARCHAR 100 — reject anything longer).
- Call CustomerController → register(). On success: set $_SESSION['customer_id'] and $_SESSION['user_role'], then redirect to views/account/my_account.php. On failure: store the error in $_SESSION['error'] and redirect back to the register page.

View — views/register.php

- A standard HTML form with action='../actions/register_action.php' method='POST'.
- Fields: Full Name, Email, Password, Country (dropdown), City, Contact Number, Address. Image upload is optional at sign-up (NULL default).
- Display $_SESSION['error'] at the top of the form if it is set, then unset it.

JavaScript — js/validate.js

- Attach a submit event listener to the registration form.
- Validate with regular expressions before the form submits:

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const phoneRegex = /^[0-9+\-\s]{7,15}$/;
```

- Show inline error messages next to each invalid field. Prevent form submission if any field fails.
- Optionally show a loading state on the submit button when clicked.

> **Follow the MVC pattern:** The View (form), the JS (client validation), the Action (server entry point), and the Model (DB logic).

TASK 4: CUSTOMER LOGIN | Due: Week 2 – 28th Sept

Objective

Build the login flow — including session management and access control that protects pages across the whole site.

Model — classes/CustomerClass.php

- Add method getCustomerByEmail($email): SELECT * FROM customer WHERE customer_email = ? — returns one row or false.
- Add method login($email, $pass): call getCustomerByEmail(), then use password_verify($pass, $row['customer_pass']) to check the hash. Return the customer row on success, false on failure.

Controller — controllers/CustomerController.php

- Add login($email, $pass) method — calls Model login(). Returns the customer array or an error array.

Action — actions/login_action.php

- POST only. Sanitise inputs. Call CustomerController → login().
- On success: store customer_id, customer_name, customer_email, user_role in $_SESSION. Redirect to index.php.
- On failure: store error in $_SESSION['error'], redirect back to login page.

View — views/login.php

- Simple form: Email + Password. Display any $_SESSION['error'].
- Below the form: link to Register page.

Access control in core/core.php

- Add is_logged_in(): returns true if $_SESSION['customer_id'] is set.
- Add is_admin(): returns true if $_SESSION['user_role'] === 1.
- Add require_login(): calls is_logged_in() — if false, redirects to login page.
- Add require_admin(): calls is_admin() — if false, redirects to index.php with an error.

Navigation update — views/layout/header.php

- Show Register | Login in the nav when not logged in.
- Show Welcome [name] | My Account | Logout when logged in.

> **MVC:** is_logged_in() and is_admin() live in core.php because they are shared utilities, not business logic. Every admin view calls require_admin() at the very top — before any HTML is output.

TASK 5: ADD BRAND | Due: Week 3

Objective

Build the first admin feature — adding a product brand. This establishes the pattern all admin tasks will follow.

Model — classes/ProductClass.php

- Create ProductClass extending Database.
- Method addBrand($name): INSERT INTO brands (brand_name) VALUES (?) — prepared statement.
- Method getAllBrands(): SELECT * FROM brands ORDER BY brand_name ASC. Returns array of rows.

Controller — controllers/ProductController.php

- Instantiate ProductClass. Expose addBrand($name) and getAllBrands() methods.

Action — actions/add_brand_action.php

- Require require_admin() at the top.
- POST only. Sanitise brand name. Call ProductController → addBrand().
- On success: set $_SESSION['success'] = 'Brand added.' Redirect to views/admin/brand.php.

View — views/admin/brand.php

- require_admin() at the top — before any HTML.
- Display a simple form: one text input for brand name, submit button.
- Below the form: fetch all brands via ProductController → getAllBrands() and display as a table with an Edit button per row.
- Show $_SESSION['success'] or $_SESSION['error'] messages.

Navigation: The Brand menu item in header.php must only appear when is_admin() returns true. Use: <?php if (is_admin()): ?>
...
<?php endif; ?>

TASK 6: EDIT BRAND | Due: Week 3

Model — classes/ProductClass.php

- Add getBrandById($id): SELECT * FROM brands WHERE brand_id = ? — returns one row.
- Add updateBrand($id, $name): UPDATE brands SET brand_name = ? WHERE brand_id = ? — prepared statement.

Action — actions/update_brand_action.php

- require_admin(). POST only. Validate brand_id (must be a positive integer) and brand_name.
- Call ProductController → updateBrand(). Redirect to views/admin/brand.php with a success message.

View — views/admin/brand.php (update)

- When a GET parameter ?edit_id=N is present, load that brand's data into the form via getBrandById().
- The form action and hidden fields change to target update_brand_action.php.
- You may reuse the same brand.php file for both add and edit, or create a separate edit_brand.php — either is acceptable provided the navigation is clear.

MVC: The brand id comes from $_GET['edit_id'] — the View reads it and pre-fills the form. The update logic lives in the Action + Controller + Model. The View never runs the UPDATE query.

TASK 7: ADD CATEGORY | Due: Week 3

Model — classes/ProductClass.php

- Add addCategory($name): INSERT INTO categories (cat_name) VALUES (?).
- Add getAllCategories(): SELECT * FROM categories ORDER BY cat_name ASC.

Action — actions/add_category_action.php

- require_admin(). POST only. Sanitise. Call ProductController → addCategory(). Redirect with success message.

View — views/admin/category.php

- require_admin() at top. Form for adding category name.
- Table of all categories with Edit buttons.
- Update nav: show Category link for admin users.

Reuse the pattern: This task is intentionally almost identical to Task 5. The goal is to practice the MVC flow a second time so it becomes familiar before the complexity of products.

TASK 8: EDIT CATEGORY | Due: Week 4

Model — classes/ProductClass.php

- Add getCategoryById($id) and updateCategory($id, $name) — mirror the brand methods from Task 6.

Action — actions/update_category_action.php

- require_admin(). POST only. Validate id and name. Update and redirect.

View — views/admin/category.php (update)

- Same ?edit_id=N pattern as brand edit.

TASK 9: PRODUCT MANAGEMENT (Add & Edit) | Due: Week 4

Objective

Add the most complex admin form so far — product entry includes a file upload and foreign-key dropdowns.

Model — classes/ProductClass.php

- Add addProduct($cat, $brand, $title, $price, $desc, $image_filename, $keywords): INSERT prepared statement.
- Add updateProduct($id, $cat, $brand, $title, $price, $desc, $image_filename, $keywords): UPDATE prepared statement.
- Add getProductById($id): SELECT with JOIN to return category name and brand name alongside product fields.

Controller — controllers/ProductController.php

- Expose addProduct() and updateProduct() — handle the image filename string (the actual file move happens in the Action).

Action — actions/add_product_action.php

- require_admin(). POST only.
- Handle image upload:

```
$allowed = ['image/jpeg', 'image/png', 'image/gif', 'image/webp'];
if (!in_array($_FILES['product_image']['type'], $allowed)) { /* reject */ }
$filename = uniqid() . '_' . basename($_FILES['product_image']['name']);
move_uploaded_file($_FILES['product_image']['tmp_name'], '../images/products/' .
$filename);
```

- Pass $filename to ProductController → addProduct(). Redirect with success.

Action — actions/update_product_action.php

- require_admin(). POST only. If a new image is uploaded, move it and update the filename; otherwise keep the existing filename from a hidden field.

View — views/admin/product.php

- require_admin() at top.
- Form fields: Product Title, Price, Description, Keywords.
- Category dropdown: loop getAllCategories() — value is cat_id, label is cat_name.
- Brand dropdown: loop getAllBrands() — value is brand_id, label is brand_name.
- Image file input (enctype='multipart/form-data' on the form).
- For edit mode (?edit_id=N): pre-fill all fields from getProductById(). Show current image preview.

> ⚠ **SECURITY:** Validate file type and size on BOTH the JavaScript side (for UX) and the PHP side (for security). Never trust client-only file validation.

TASK 10: PRODUCT DISPLAY (List & Search) | Due: Week 5

Objective

Build the customer-facing product pages — home page product grid, single product detail, category/brand filtering, and search results.

Model — classes/ProductClass.php

- Add getFeaturedProducts($limit = 6): SELECT with ORDER BY RAND() LIMIT ? — mirrors original getPro().
- Add getProductsByCategory($cat_id): SELECT WHERE product_cat = ?.
- Add getProductsByBrand($brand_id): SELECT WHERE product_brand = ?.
- Add searchProducts($query): SELECT WHERE product_title LIKE ? OR product_keywords LIKE ? — use '%' . $query . '%' in a prepared statement.
- All methods return an array of rows (empty array if no results — never false).

View — views/home.php

- Include views/layout/header.php and views/layout/sidebar.php.

- Check for GET params: if ?cat=N call getProductsByCategory(); if ?brand=N call getProductsByBrand(); otherwise call getFeaturedProducts().
- Loop the results — for each product display: image, title, price, Details link (?pro_id=N), Add to Cart link (?add_cart=N).
- If the result array is empty display a friendly 'No products found' message.

View — views/single_product.php

- Require a valid ?pro_id=N GET param — redirect home if missing or invalid.
- Call getProductById() and display: image, title, category, brand, price, description, keywords, Add to Cart.

View — views/search_results.php

- Read ?user_query from GET. Call searchProducts(). Display results in the same card format as home.php.

View — views/layout/sidebar.php

- Call getAllCategories() and getAllBrands() via ProductController.
- Render as <ul> lists with links: Category Name.

> **MVC:** The View files call Controller methods to get data — they never call Model methods directly. views/home.php calls $controller->getFeaturedProducts(), not $model->getFeaturedProducts(). The Controller is always the middleman.

TASK 11: ADD TO CART | Due: Week 6

Objective

Implement the Add to Cart flow, including duplicate detection and the cart summary in the header.

Model — classes/CartClass.php

- Create CartClass extending Database.
- Add isInCart($product_id, $ip): SELECT from cart WHERE p_id = ? AND ip_add = ? — returns true/false.
- Add addToCart($product_id, $qty, $ip, $customer_id = null): INSERT INTO cart (p_id, ip_add, qty) VALUES (?, ?, ?) — prepared statement.
- Add getCartItems($ip): SELECT cart.*, products.product_title, products.product_price, products.product_image FROM cart JOIN products ON cart.p_id = products.product_id WHERE cart.ip_add = ?.
- Add getCartCount($ip): SELECT COUNT(*) FROM cart WHERE ip_add = ?. Returns integer.
- Add getCartTotal($ip): Iterate cart items, SUM(qty * product_price). Returns float.

Controller — controllers/CartController.php

- Instantiate CartClass. Expose addToCart(), getCartItems(), getCartCount(), getCartTotal().

Action — actions/add_to_cart_action.php

- Read ?add_cart=N from GET (links trigger this, matching the original pattern).
- Get client IP via get_ip() from core.php.
- Call CartController → isInCart(). If already in cart: redirect with message 'This item is already in your cart. Go to cart to update quantity.'
- Otherwise: call addToCart(). Redirect back to the referring page.

View — views/layout/header.php (update)

- Add cart summary bar: 'Shopping Cart — Total Items: N | Total Price: $X | Go to Cart'.
- Call CartController → getCartCount() and getCartTotal() here.
- Show Login link if not logged in; Logout link if logged in.

View — views/cart.php

- Call CartController → getCartItems(). Display as a table: Remove checkbox, Product name + image, Qty input, Unit price.
- Show subtotal row.
- Buttons: Remove from Cart (POST form), Update Quantity (POST form), Continue Shopping, Checkout.

Guest carts: Cart rows are tied to IP address — guests can shop without an account. If the customer registers/logs in during a session, their existing cart (by IP) carries over automatically since the IP stays the same.

TASK 12: CART MANAGEMENT | Due: Week 6

Objective

Complete the cart — remove items and update quantities.

Model — classes/CartClass.php

- Add removeFromCart($product_id, $ip): DELETE FROM cart WHERE p_id = ? AND ip_add = ? — prepared statement.
- Add updateQty($product_id, $new_qty, $ip): UPDATE cart SET qty = ? WHERE p_id = ? AND ip_add = ?. Validate that new_qty >= 1.

Action — actions/remove_from_cart_action.php

- POST only. Validate product_id is a positive integer. Call CartController → removeFromCart(). Redirect to views/cart.php.

Action — actions/update_qty_action.php

- POST only. Loop through submitted qty[] and update[] arrays. Validate each quantity is a positive integer before calling updateQty(). Redirect to views/cart.php.

View — views/cart.php (update)

- Remove form: checkboxes with name='remove[]' value=product_id, submit to remove_from_cart_action.php.
- Update form: text inputs name='qty[]', hidden name='update[]' value=product_id, submit to update_qty_action.php.
- Continue Shopping button navigates to index.php. Checkout button links to views/checkout.php.

MVC: Two separate Action files handle two separate operations (remove vs update). Each Action does one thing, calls one Controller method, then redirects. This keeps the cart View clean — it only displays data.

TASK 13: PAYMENT INTEGRATION | Due: Week 7

Objective

Implement Paystack payment, record the order, email the customer, and display the invoice.

Model — classes/CartClass.php

- Add processPayment($customer_id, $ip, $reference, $amount): run the following in a DB transaction —
- 1. Generate invoice number: mt_rand(100000, 999999).
- 2. INSERT into orders table (customer_id, invoice_no, invoice_no, order_date, order_status='paid', created_at).
- 3. Copy cart rows into orderdetails table (order_id, product_id, qty).
- 4. DELETE from cart WHERE ip_add = ?.
- 5. Return the new order_id.
- Add getOrderById($order_id): SELECT order with joined order_details and product names.

Action — actions/process_payment_action.php

- Called by Paystack's success callback or your own payment form submission.
- Verify the Paystack transaction server-side using Paystack's Verify API endpoint before recording the order — never trust a client-side success callback alone.

```
$url = 'https://api.paystack.co/transaction/verify/' . $reference;
$headers = ['Authorization: Bearer sk_test_YOUR_KEY'];
// Use cURL to call the Paystack API and decode the JSON response
```

- Only call processPayment() if the Paystack API confirms the transaction was successful.
- On success: redirect to views/payment_success.php?order_id=N.
- On failure: redirect to views/payment_failed.php with an error message.

View — views/payment.php

- Display cart summary: products, quantities, unit prices, total.
- Include the Paystack inline JS and initialise with customer email and total amount (in kobo/pesewas — multiply by 100):

```
<script src='https://js.paystack.co/v1/inline.js'></script>
<script>
  function payWithPaystack() {
    var handler = PaystackPop.setup({
      key: 'pk_test_YOUR_PUBLIC_KEY',
      email: '<?= htmlspecialchars($_SESSION["customer_email"]) ?>',
      amount: <?= $total * 100 ?>,
      currency: 'GHS',
      onSuccess: function(response) {
        window.location = 'actions/process_payment_action.php?ref=' +
response.reference;
      },
      onCancel: function() { alert('Payment cancelled.'); }
    });
    handler.openIframe();
  }
</script>
```

View — views/payment_success.php

- Read ?order_id from GET. Call CartController → getOrderById(). Display the full invoice: order number, products, quantities, prices, total.
- Optionally send a confirmation email using PHP's mail() function or a library like PHPMailer.

> ⚠ **SECURITY:** Always verify payment server-side via the Paystack API before marking an order as paid. A user could manually browse to process_payment_action.php?ref=fake without paying.

> **Paystack docs:** https://paystack.com/docs/payments/accept-payments — use test keys during development. Test card: 4084 0840 8408 4081, any future expiry, CVV 408.

APPENDIX: QUICK REFERENCE

MVC File Responsibility Cheatsheet

File type	CAN contain	MUST NOT contain
`classes/*.php (Model)`	SQL queries, data validation logic, return arrays/booleans	echo, HTML, $_POST, $_SESSION, header()
`controllers/*.php`	Instantiate Model classes, call methods, return results	SQL queries, echo, HTML output, header() redirect
`actions/*.php`	Read $_POST/$_GET, call Controller, set $_SESSION, redirect	SQL, HTML output, echo (except redirect header)
`views/*.php`	HTML, CSS, PHP loops/conditionals to display data	SQL queries, INSERT/UPDATE/DELETE, business logic
`core/core.php`	Session start, shared helpers (is_logged_in etc), config	Business logic, SQL, HTML output

JavaScript Validation Patterns

Add these to js/validate.js and attach them to all forms that require client-side validation.

```
// Email
const emailOK = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);

// Phone — 7 to 15 digits, allows +, -, spaces
const phoneOK = /^[0-9+\-\s]{7,15}$/.test(phone);

// Password — minimum 8 characters, at least one digit
const passOK = /^(?=.*\d).{8,}$/.test(password);
```

```
// Non-empty text field
const nameOK = name.trim().length >= 2;

// Attach to form
document.getElementById('register-form').addEventListener('submit', function(e) {
  if (!emailOK) { e.preventDefault(); showError('email-error', 'Invalid email'); }
});
```

Prepared Statement Pattern (PHP)

Use this pattern for every query that includes user input — no exceptions.

```
// SELECT example
$stmt = $this->conn->prepare('SELECT * FROM customer WHERE customer_email = ?');
$stmt->bind_param('s', $email);
$stmt->execute();
$result = $stmt->get_result();
$row = $result->fetch_assoc();
$stmt->close();

// INSERT example
$stmt = $this->conn->prepare(
  'INSERT INTO customer (customer_name, customer_email, customer_pass) VALUES (?, ?, ?)'
);
$stmt->bind_param('sss', $name, $email, $hash);
$stmt->execute();
$stmt->close();
```

Assessment Guidelines

- MVC separation is graded strictly: SQL in a View file = marks deducted. HTML output in a Model = marks deducted.
- All forms must have both client-side (JS regex) and server-side (PHP) validation.
- All SQL queries with user input must use prepared statements — no exceptions.
- Passwords must use password_hash() / password_verify() — plain text = zero marks for that task.
- File uploads must validate type and size on the PHP side.
- Admin pages must call require_admin() before any output — security check comes first.
- Code plagiarism is checked automatically — write your own code.

Build clean code. Be proud of what you submit.

Extra – Use AI for Recently purchased, or other forms of recommendation. Receipt and email features